

2.7 解: (1) 算法描述:

- ① 用堆排序算法对数组A由小到大进行排序.
- ② 用一个包含 $\frac{n}{2}+1$ 个元素的窗口扫描数组, 每次比较窗口最左侧与最右侧元素, 若最左侧与最右侧元素相等, 则A中存在主元素.
- ③ 反之则不存在.

时间复杂度分析:

假设最坏情况下 $W(n) = W_1(n) + W_2(n)$

其中 $T_1(n)$ 为对数组排序的时间复杂度, $T_2(m)$ 为用窗口扫描数组的时间复杂度.

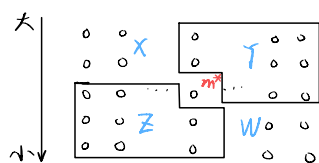
显然, $W_1(n) = O(n \log n)$

以比较为基本运算, 则扫描过程最多比较 $\frac{n}{2}$ 次
故 $W_2(n) = O(n)$

综上, $W(n) = O(n \log n)$

(2) 算法描述:

- ① 由于主元素一定是A的中位数, 故先利用分治思想求中位数:
- a. 将A中元素5个分为一组, 共 $m = \lceil \frac{n}{5} \rceil$ 组
- b. 遍历所有组, 找出每个组的中位数组成集合M
 $|M| = \lceil \frac{n}{5} \rceil$
- c. 递归调用该算法, 求出 $|M|$ 的中位数 m^* .
- d. 以 m^* 为 pivot, 遍历M, 将中位数小/大于 m^* 的组放在 m^* 所在组的左/右侧, 得到重排后的 A' .
- e. 将 A' 划分为X, Y, Z, W 4个子集合. 划分方式如下:
遍历每一组, 若该组位于 m^* 组左侧, 则将该组中小于等于该组中位数的元素加入Z, 其余加入X. 类似可得Y与W, 满足Y中元素均大于等于 m^* , Z中元素均小于等于 m^* , 如下图



- f. 遍历X, W, 将其中小于等于 m^* 的元素加入Z(Y)
- g. 若 $\lceil \frac{n}{5} \rceil \leq |Z|$, 则递归调用该算法, 在Z中找第 $\lceil \frac{n}{5} \rceil$ 小的数; 若 $\lceil \frac{n}{5} \rceil = |Z| + 1$, 则 m^* 为中位数; 若 $\lceil \frac{n}{5} \rceil > |Z| + 1$, 则在Y中找第 $\lceil \frac{n}{5} \rceil - |Z| - 1$ 小的数.

该算法的时间复杂度为 $O(n)$

- ② 找到中位数 m 后, 遍历A, 记录 m 出现的次数 k
若 $k > \frac{n}{2}$, 则 m 为主元素, 否则不存在主元素.

时间复杂度分析: 该算法 ① ② 步均为 $O(n)$, 故时间复杂度为 $O(n)$.

(3) 算法描述:

MainEle(A):

if $|A| = 1$ then 检查该元素在原数组A中出现的次数
else if $|A| = 0$ then A无主元素

else 将A中元素两两分组

比较每一组中元素, 若相等, 则保留其中一个, 加入B. 否则全部丢弃

MainEle(B) // 若A中有轮空元素, 则单独测试

证: ① A存在主元素 m , 假设 n 为偶数

设分组后含有 a 个主元素, 组数为 a , 含有 b 个主元素, c 个非主元素, 组数为 b , 含有 c 个非主元素, 组数为 c , 则有 $2a + b > b + 2c \Rightarrow a > c$, 故 $|B|$ 中 m 的个数一定大于 $\frac{|B|}{2}$, 子问题与原问题一致

若 n 为奇数, 则有在轮空的情况, 此时将轮空元素与其他元素依次比较, 记录该元素出现次数, 若次数 $> \frac{n}{2}$, 则该元素即为主元素; 否则丢弃.

- ② A不存在主元素, 即 $\forall a \in A, a$ 至多出现 $\frac{n}{2}$ 次
考虑最坏情况: B中最终只剩下1个元素, 此时遍历A检查是否为主元素即可

时间复杂度分析: 以比较作为基本运算.

$$\text{最坏情况下: } \begin{cases} W(n) = W(\frac{n}{2}) + O(n) \\ W(1) = 0 \end{cases} \Rightarrow W(n) = O(n)$$

2.12 算法描述:

- ① 对B排序

- ② 遍历A, 对 $\forall a_i, i = 1, 2, \dots, n$

在B中二分查找 a_i , 若找到则将 a_i 加入C.

伪代码:

And(A, B):

1. sort(B)
2. for $i \leftarrow 1$ to n :
3. index $\leftarrow$ BinarySearch(B, a_i)
4. if index ≥ 1 then
5. add(C, a_i)

时间复杂度:

$$\begin{aligned} T(n) &= O(n \log m) + O(n \cdot \log m) \\ &= O(n \log m) = O(n \log \log n) \end{aligned}$$

2.15 解: (1) A: $W(n) = i \times O(n) = O(in) = O(n^{\frac{1}{2}})$ ($i \leq n^{\frac{1}{2}}$)
 B: $W(n) = O(n \lg n) + O(i) = O(n \lg n) + O(n^{\frac{1}{2}})$
 $= O(n \lg n)$

(2) 算法描述:

- ① 利用快速选择算法找到第 $i+1$ 大的元素 m^*
- ② 排序在①中对 S 划分得到的大于 m^* 的元素集合
 并从小到大输出

伪代码:

```

Select(S, k): // 求第 k 大数
    将 S 中元素 5 个一组分为  $n_m = \lceil \frac{n}{5} \rceil$  组,  $S_1, S_2, \dots, S_{n_m}$ 
    for  $j \leftarrow 1$  to  $n_m$ :
        求  $S_j$  中位数  $m_j$ , 将  $m_j$  加入集合 M
     $m^* \leftarrow \text{Select}(M, \lceil \frac{|M|}{5} \rceil)$  // 求 M 中位数
    划分 S // 分析见前
    遍历, 得  $S_1, S_2$ , 其中  $\forall s \in S_1, s \leq m^*$ ,
         $\forall s \in S_2, s \geq m^*$ .
    if  $k = |S_2| + 1$  then 输出  $m^*, S_2$ 
    else if  $k \leq |S_2|$  then Select( $S_2, k$ )
    else Select( $S_1, k - |S_2| - 1$ )
    
```

PrintMax(S, i):

```

(boundary,  $S_2$ )  $\leftarrow$  Select(S,  $i+1$ )
sort( $S_2$ ) // 从大到小排序
for  $j \leftarrow 1$  to  $i$ :
    print( $S_2[j]$ )
    
```

时间复杂度分析:

$$\begin{aligned}
 W(n) &= W_1(n) + W_2(n) \\
 &= \theta(n) + O(i \lg i) \\
 &= \theta(n) + O(n^{\frac{1}{2}} \lg n) \\
 &= \theta(n)
 \end{aligned}$$

2.16 解: (1) 该问题等价于寻找 S 中的最大数和最小数
 故可通过一次遍历找到满足条件的 x 和 y
 时间复杂度为 $O(n)$

(2) 该问题等价于寻找坐标轴上距离最近的点对。
 考虑用分治。

算法描述:

MinDistance(S)

- ① 若 $|S| \leq 3$, 直接求出 S 中距离最近的点数
- ② 以 $x = l$ 为基准, 遍历 S , 将 S 划分为大小 $\frac{n}{2}$ 和 $\frac{n}{2}$ 相近的 L 和 R , 其中 L 中元素小于等于 l , R 中元素大于 l $O(n)$
- ③ $\delta_L \leftarrow \text{MinDistance}(L)$
 $\delta_R \leftarrow \text{MinDistance}(R)$
- ④ 遍历 L 和 R , 求出 L 两侧距 l 最近的数 l^*, r^*
 $\delta \leftarrow \min\{\delta_L, \delta_R, r^* - l^*\}$ $O(n)$

时间复杂度:

$$\begin{aligned}
 T(n) &= 2T(\frac{n}{2}) + O(n) \\
 \Rightarrow T(n) &= \theta(n \lg n)
 \end{aligned}$$

2.18 解: 算法描述:

- ① 遍历所有点, 计算每个点到原点的距离并记录
 点与距离间的一一映射关系 (由题知)
- ② 用快速选择算法求第 $(\lfloor \sqrt{n} \rfloor + 1)$ 小的距离与小于该距离的距离集合 S_1 (分析见前)
- ③ 对 S_1 排序后从大到小输出与距离对应的点的
 标号

伪代码:

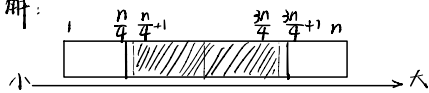
```

for  $i \leftarrow 1$  to  $n$ :
    Dis[i] = ( $\sqrt{x_i^2 + y_i^2}, i$ ) // i 用于记录点的标号
 $S_1 \leftarrow \text{Select}(\text{Dis}, \lfloor \sqrt{n} \rfloor + 1)$ 
sort( $S_1$ ) // 按照距离逆序排序
for  $i \leftarrow 1$  to  $\lfloor \sqrt{n} \rfloor$ :
    输出  $S_1[i].\text{point\_index}$ 
    
```

时间复杂度:

$$\begin{aligned}
 T(n) &= T_1(n) + T_2(n) + T_3(n) + T_4(n) \\
 &= O(n) + O(n) + O(\lfloor \sqrt{n} \rfloor \lg \lfloor \sqrt{n} \rfloor) \\
 &\quad + O(\lfloor \sqrt{n} \rfloor) \\
 &= O(n)
 \end{aligned}$$

2.23 解:



如上图所示, 问题等价于求一个在 A 中阴影部分区域内的函数.

算法描述: 利用快速选择算法求出 A 中第 $\frac{n}{4}$ 小和第 $\frac{3n}{4}+1$ 小的函数 l 和 r .

若 $l = r$ 则不存在近似中值.

若 $l \neq r$, 遍历 A , 若找到 x , $l < x < r$ 则 x 为 A 的近似中值, 否则 A 不存在近似中值.

时间复杂度: $T(n) = T_1(n) + T_2(n) = O(n) + O(n) = O(n)$

2.27 解: 算法描述:

设 n 个商店的坐标分别为 (x_i, y_i) , $i = 1, 2, \dots, n$

仓库的坐标为 x, y .

则总路程为 $\sum_{i=1}^n |x - x_i| + \sum_{i=1}^n |y - y_i|$

① n 为奇数, 则取 x, y 分别为 $\{x_i\}, \{y_i\}$, $i = 1, 2, \dots, n$ 中位数即可

② n 为偶数, 则取 x_1, x_n 和 y_1, y_n 分别为 $\{x_i\}, \{y_i\}$, $i = 1, 2, \dots, n$ 的两个中位数

分别计算 $(x_1, y_1), (x_n, y_1), (x_1, y_n), (x_n, y_n)$

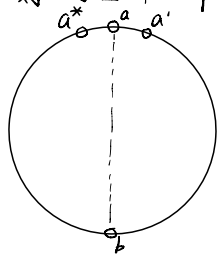
到各商店的路程, 以路程最短两点作为仓库位置.

时间复杂度: $T(n) = O(n)$

补充题 1.

解: 利用二分查找, 算法如下:

在环上找一对“对径点”, 即在环上相距最远的一对点. 如下图:



依题意: $a \neq b$. 不妨设 $a < b$.

考虑与 a 相邻的两元素 a^* 与 a'

① $a < a^* \wedge a < a'$, 则 a 为极小值, 算法结束

② $a > a^* \vee a > a'$. 不妨设 $a > a'$

则有 $a' < a < b$, 从而在 $\widehat{a'b}$ 上必有极小值 (不包括 b), 若 $\widehat{a'b}$ 间无其他元素, 则极小值为 a'

算法结束

③ 在 a 与 b 间连边形成环 $aa'b$. 在该环上重复以上过程

时间复杂度: $T(n) = T(\frac{n}{2}) + O(1)$

$= O(\log n)$

补充题 1.

将环展开为数组 $A[0 \dots n-1]$.

$\Delta \leftarrow A[0] - A[n-1]$

for $i \leftarrow 0$ to $n-1$:

pre- $\Delta \leftarrow \Delta$

$\Delta \leftarrow A[(i+1) \% n] - A[i]$

if $\Delta > 0$ and pre- $\Delta < 0$ then

return $A[i]$ // 必然能找到

时间复杂度: $O(n)$

补充题 2.

解: 基本思路: 将轮空的芯片设法也加入测试流程

只需将轮空的芯片记录并保留到下一轮, 其余测试保持不变.

当测试到最后剩余一个芯片时, 该芯片为好芯片

若最终所有芯片均被淘汰, 则最后一次记录到的被轮空芯片为好芯片.

算法正确性证明如下:

① 循环不变量: 任意一次测试后好芯片数 $\geq$ 坏芯片数均成立

a. 芯片数为偶数

设 a, b, c 分别为好-好, 好-坏, 坏-坏组数

$\Rightarrow 2a + b \geq b + 2c$

$\Rightarrow a \geq c$

b. 芯片数为奇数

此时好芯片数 $\geq$ 坏芯片数 + 1

取一芯片轮空, 余下芯片可化归为情况 a.

② 若最终只剩一个芯片

由循环不变量知: 该芯片必为好芯片

③ 若最终不剩芯片

则分组情况均为好-坏, 好芯片数 = 坏芯片数

而每一个好芯片均来自于好-好, 坏芯片同理

因此在最后一次出现轮空时, 分组情况一定为好-好 k 组, 坏-坏 k 组, 好-坏若干组

$\Rightarrow$ 好芯片数 = 坏芯片数

$\Rightarrow$ 被轮空的是好芯片.